Computer Organization and Architecture	
Lab #:	**7**
Lab Title:	Exception Handling in MIPS
Submitted by:	
Names	**Registration #**
Sumaira Safeer	FA22-BCE-019

Rubrics name & number					**Marks**	
					In-Lab	**Post-Lab**
Engineering Knowledge	***R2: Use of Engineering Knowledge and follow Experiment Procedures:*** *Ability to follow experimental procedures, control variables, and record procedural steps on lab report.*					
Problem Analysis	***R6: Experimental Data Analysis:*** *Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.*					
Design	***R8: Best Coding Standards:*** *Ability to follow the coding standards and programming practices.*					
Modern Tools Usage	***R9: Understand Tools:*** *Ability to describe and explain the principles behind and applicability of engineering tools.*					
Individual and Teamwork	***R12: Individual Work Contributions:*** *Ability to carry out individual responsibilities.*					
	R13: Management of Team Work: *Ability to appreciate, understand and work with multidisciplinary team members.*					
Rubrics #	**R2**	**R6**	**R8**	**R9**	**R12**	**R13**
In Lab						
Post- Lab						

LAB: 07

Task No: 01

Exception Handling in MIPS .

Code:

```
#############################################################################
######
# USER TEXT SEGMENT
#
# MARSstarts execution at label main in the user .text segment.
#############################################################################
######
.globl main
.text

main:
 # The largest 32 bit positive two's complement number.
li $s0, 0x7fffffff

# Trigger an arithmetic overflow exception.
      addi $s1, $s0, 1

# Trigger a bad data address (on load) exception.
lw $s0, 0($zero)

# Trigger a trap exception.
teqi $zero, 0

todo_3:
# Enable keyboard interrupts.
```

```
# Get value of thememory-mapped receiver control register.
lw $s0, 0xffff0000

# Set bit 1 (interrupt enable) in receiver control to 1.
ori $s1, $s0, 0x2    # Enable interrupt by setting bit 1

# Update the memory-mapped receiver control register.
sw $s1, 0xffff0000

infinite_loop:
# Simulate the CPU doing something useful.
addi $s0, $s0, 1
j infinite_loop

#############################################################################
######
# KERNEL DATA SEGMENT
#############################################################################
######

.kdata

UNHANDLED_EXCEPTION:
UNHANDLED_INTERRUPT:
OVERFLOW_EXCEPTION:

.asciiz "===>      Unhandled exception       <===\n\n"
.asciiz "===>      Unhandled interrupt       <===\n\n"
.asciiz"===>      Arithmetic overflow       <===\n\n"

TRAP_EXCEPTION:
```

```
.asciiz "===>        Trap exception        <===\n\n"
BAD_ADDRESS_EXCEPTION:.asciiz "===>   Bad data address exception   <===\n\n"

#########################################################################
######
# KERNEL TEXT SEGMENT
#########################################################################
######
# The exception vector address for MIPS32.
.ktext 0x80000180

__kernel_entry_point:
# Get value in cause register.
mfc0 $k0, $13

# Mask all but the exception code (bits 2-6) to zero.
andi $k1, $k0, 0x00007c

# Shift two bits to the right to get the exception code.
srl  $k1, $k1, 2

# The exception code is zero for an interrupt and non-zero for allexceptions.
beqz $k1, __interrupt

__exception:
# Branch based on the exception code in $k1.
beq $k1, 12, __overflow_exception

todo_2:
# Branch to __bad_address_exception for exception code 4.
beq $k1, 4, __bad_address_exception
```

```
# Branch to__trap_exception for exception code 13.
beq $k1, 13, __trap_exception

__unhandled_exception:
# Print error message for unhandled exceptions.
li $v0, 4
la $a0, UNHANDLED_EXCEPTION
syscall
j __resume_from_exception

__overflow_exception:
# Print error message for overflow exception.
li $v0, 4
la $a0, OVERFLOW_EXCEPTION
syscall
j __resume_from_exception

__bad_address_exception:
# Print error message for bad address exception.
li $v0, 4
la $a0, BAD_ADDRESS_EXCEPTION
syscall
j __resume_from_exception

__trap_exception:
# Print error message for trap exception.
li $v0, 4
la $a0, TRAP_EXCEPTION
syscall
j __resume_from_exception
```

```
__interrupt:
# Value of cause register should already be in $k0.
# Mask all but bit 8 (interrupt pending) to zero.
andi $k1, $k0, 0x00000100

# Shift 8 bits to the right to get the interrupt pending bit as LSB.
srl  $k1, $k1, 8

# Branch on the interrupt pending bit.
beq  $k1, 1, __keyboard_interrupt

__unhandled_interrupt:
# Print error message for unhandled interrupt.
li $v0, 4
la $a0, UNHANDLED_INTERRUPT
syscall
j __resume

__keyboard_interrupt:
# Store content of the memory-mapped receiver data register in $k1.
lw $k1, 0xffff0004
# Print the character fromreceiver data.
move $a0, $k1
li $v0, 11
syscall
j __resume

__resume_from_exception:
# Get the value of EPC.
mfc0 $k0, $14
```

todo_1:

```
# Skip offending instruction by adding 4 to the EPC value.
addi $k0, $k0, 4
# Update EPC in coprocessor 0.
mtc0 $k0, $14
```

__resume:

```
# Use eret to set the PC to the value saved in the EPC register.
eret
```

Output: